Mestrado em Engenharia Informática
Gestão de Identidade

Semana 3 — Login with Google (OIDC)

Ricardo Costa
rcosta@estg.ipp.pt
Escola Superior de Tecnologia e Gestão do Instituto Politécnico do Porto

Objetivos da Semana 3

No final da semana os estudantes deverão conseguir:

- Explicar o fluxo **OpenID Connect** com palavras próprias
- Implementar um **Login with Google** funcional
- Distinguir autenticação (OIDC) de autorização (OAuth2)
- Validar claims básicas de segurança no `id_token`
- Compreender os principais termos técnicos (PKCE, token, claims)

Agenda (4h)

- **Teoria explicativa (1h30)**
  - O problema que OIDC resolve
  - Fluxo Authorization Code + PKCE
  - Tokens e claims
  - Riscos comuns e controlos mínimos
- **Prática guiada (2h30)**
  - Implementar Login with Google
  - Ver perfil autenticado
  - Validar `iss`, `aud`, `exp`

O problema de autenticação moderna

Sem federação, cada aplicação:

- guarda password local
- implementa login próprio
- aumenta superfície de ataque

Com OIDC:

- login é delegado no IdP
- app recebe identidade validada
- menos responsabilidade direta sobre passwords

Conceitos essenciais

- **IdP (Identity Provider):** autentica utilizador (Google, depois Keycloak)
- **RP (Relying Party):** aplicação que confia no IdP
- **OIDC:** identidade sobre OAuth2
- **Claim:** informação sobre o utilizador/token

OAuth2 vs OIDC (sem confusão)

- **OAuth2** -> autorização (acesso a recursos)
- **OIDC** -> autenticação (quem é o utilizador)

Regra prática:

- Quero login -> OIDC
- Quero chamar API em nome do utilizador -> OAuth2

Onde entra o SAML?

SAML 2.0 é um padrão de federação e SSO muito usado em ambientes enterprise.

Características típicas:

- foco em autenticação federada web
- troca de mensagens em XML (assertions)
- comum em integrações corporativas e sistemas legados

SAML vs OIDC vs OAuth2

- **SAML**: SSO/federação tradicional enterprise (XML)
- **OIDC**: autenticação moderna para apps web/mobile (JSON/JWT)
- **OAuth2**: autorização delegada para acesso a APIs

Resumo rápido:

- SAML e OIDC respondem ao "quem é o utilizador"
- OAuth2 responde ao "o que esta app pode aceder"

Fluxo recomendado: Authorization Code + PKCE

1. Utilizador abre app (RP)
2. RP redireciona para Google
3. Utilizador autentica no Google
4. Google devolve `code`
5. RP troca `code` por tokens
6. RP valida `id_token` e cria sessão

O que é PKCE?

PKCE (*Proof Key for Code Exchange*) é um mecanismo de proteção do fluxo Authorization Code.

Objetivo:

- impedir que terceiros usem um `code` intercetado

Como funciona (resumo):

- a app gera um segredo temporário (`code_verifier`)
- envia uma versão derivada (`code_challenge`) no pedido inicial
- no fim, só troca o `code` por tokens se apresentar o `code_verifier` correto

> Atualmente deve ser padrão, mesmo em apps web simples.

O que é um token?

Um **token** é um artefacto digital emitido pelo IdP/Authorization Server que transporta prova de identidade ou autorização.

Tipos no fluxo:

- `id_token`: prova de identidade do utilizador
- `access_token`: permite acesso a recursos/APIs
- `refresh_token`: permite renovar tokens (opcional)

Para esta aula, foco principal:

- `id_token`

Claims importantes no id_token

- `iss`: emissor (quem assinou)
- `aud`: destinatário (client_id esperado)
- `sub`: identificador estável do utilizador
- `exp`: expiração
- `email`, `name`, `picture` (perfil)

Validações mínimas obrigatórias

App deve validar:

- `iss` esperado
- `aud` = client da app
- `exp` ainda válido
- assinatura do token (biblioteca OIDC trata isto)

Erros comuns em implementações académicas

- confundir OAuth2 com OIDC
- usar redirect URI errada
- aceitar token sem validar `aud`
- logar tokens em texto limpo
- scopes excessivos sem necessidade

Segurança mínima para esta aula

- scopes mínimos: `openid email profile`
- redirect URI exata (`http://localhost:3000/auth/callback`)
- não commitar client secret
- sessão curta e logout funcional

Resultado esperado da prática

Cada grupo deve mostrar:

- botão/caminho de Login with Google
- utilizador autenticado na app
- claims principais visíveis
- validação básica explicada

Transição para prática

Vamos implementar uma app mínima com:

- rota `/login`
- callback `/auth/callback`
- página protegida `/profile`
- logout `/logout`

Prática — organização

- Setup (30 min)
- Configuração Google Cloud (30 min)
- Implementação app (50 min)
- Testes + validações (30 min)
- Fecho (10 min)

Prática — pré-requisitos

- Node.js 18+
- Conta Google
- Navegador

Comandos iniciais:

```
mkdir gi-semana3-login-google
cd gi-semana3-login-google
npm init -y
npm install express express-session openid-client@5 dotenv
```

Prática — Google Cloud (passo a passo)

1. Ir a https://console.cloud.google.com/
2. Criar projeto
3. Configurar **OAuth consent screen**
4. Criar **OAuth Client ID** (Web application)
5. Definir redirect URI:

```
http://localhost:3000/auth/callback
```

6. Guardar `CLIENT_ID` e `CLIENT_SECRET`

Prática — package.json (ES Modules)

Para evitar erro `Cannot use import statement outside a module`, garantir no `package.json`:

```
1 {
2   "type": "module"
3 }
```

Prática — ficheiro .env

Criar `.env`:

```
PORT=3000
SESSION_SECRET=trocar-por-um-segredo-forte
OIDC_ISSUER=https://accounts.google.com
OIDC_CLIENT_ID=COLOCAR_AQUI
OIDC_CLIENT_SECRET=COLOCAR_AQUI
OIDC_REDIRECT_URI=http://localhost:3000/auth/callback
```

Prática — código base da app

Usar o ficheiro base disponibilizado:

- `semana3-app-base.js`

Guardar como `app.js` na pasta do projeto.

> Evitamos copy/paste longo e focamos no entendimento do fluxo.

Prática — executar e testar

Executar:

```
node app.js
```

Abrir `http://localhost:3000` e validar:

- login funciona
- callback sem erro
- `/profile` mostra claims
- logout limpa sessão

Prática — troubleshooting

- `Cannot use import statement outside a module` -> adicionar `"type": "module"` no `package.json`
- `openid-client does not provide an export named 'Issuer'` -> instalar `openid-client@5`
- `redirect_uri_mismatch` -> URI no Google Cloud não coincide
- `invalid_client` -> client id / secret errados
- `state mismatch` -> sessão perdida a meio do fluxo

Mini-quiz rápido

1. OIDC resolve autenticação ou autorização?
2. Para que serve o `aud`?
3. Porque usar PKCE?
4. Qual diferença entre `id_token` e `access_token`?

Respostas mini-quiz

1. Autenticação
2. Garantir que o token é para esta app
3. Reduzir risco na troca do authorization code
4. `id_token` identifica utilizador; `access_token` autoriza acesso a recursos

Fecho

Mensagem-chave:

> Se dominar OIDC e validação de tokens, consegue trocar de IdP sem reaprender o essencial.